근데 2차 과제에선 리스트랑 조건문 활용해서 투두리스트만들고 3차에서 기분 넣어서 행운의 색 입력되는거+중요도 표시 4차에서 이거 반복..+모듈화 하고싶어 너무 간당하면 뭐 더 추가할 거 있는지 적어줘 어떤 기능 넣으면 더 좋을까?

학생분의 계획을 들어보니 단계별 확장성이 매우 좋습니다. 다만, 교수님께서 보시기에 각 차수별 분량이 적절하고 '성장'하는 느낌을 주려면 몇 가지 디바이스를 더 추가하는 것이 좋습니다.

제시해주신 로드맵을 바탕으로, **전공자 수준의 디테일을 살릴 수 있는 추가 기능**들을 제안해 드립니다.

🚀 수정된 단계별 기능 확장 로드맵

🟢 2차 과제: 기초 구조 잡기 (리스트 + 조건문)

- **현재 계획:** 할 일 입력 및 저장 (단일 혹은 중첩 리스트)
- **추가하면 좋은 것: "중요도 기반 필터링"**
  - 단순히 저장만 하는 게 아니라, `if` 문을 활용해 "중요도가 4점 이상인 급한 일만 보기" 혹은 "공부 카테고리만 보기" 기능을 넣어보세요.

부 카테고리만 보기" 기능을 넣어보세요.

- **기술 포인트:** 리스트 인덱싱(`todo[0]`), `if-elif-else` .

🟡 **3차 과제: 사용자 경험 및 흐름 제어 (기분 + 반복문 + 메뉴)**

- **현재 계획:** 오늘의 기분 입력 → 행운의 색 출력 + 중요도 표시
- **추가하면 좋은 것: "무한 루프 메뉴 시스템"** 및 **"할 일 완료/삭제"**
  - 프로그램이 한 번 실행되고 끝나는 게 아니라, `while True` 를 이용해 `1. 할 일 추가 2. 목록 보기 3. 기분 점검 4. 종료` 메뉴를 만듭니다.
  - 기분 입력 시 `random` 모듈을 살짝 써서 '오늘의 조언'을 무작위로 나오게 하면 점수가 더 올라갑니다.
  - **기술 포인트:** `while` , `break` , `random.choice()` .

🔴 **4차 과제: 고도화 및 구조화 (함수 + 딕셔너리 + 예외처리)**

- **현재 계획:** 모듈화 (함수 나누기)
- **추가하면 좋은 것: "데이터 통계"** 및 **"파일 저장(선택)"**
  - **통계:** 현재 리스트에 남은 할 일 개수, 완료율 등을 계산하는 함수.

- **통계:** 현재 리스트에 남은 할 일 개수, 완료율 등을 계산하는 함수.
- **딕셔너리:** 리스트 대신 `{'할일': '파이썬공부', '중요도': 5}` 식으로 자료구조 고도화.
- **예외처리:** 숫자를 넣어야 할 중요도에 글자를 넣었을 때 튕기지 않게 `try-except` 처리.
- **기술 포인트:** `def`, `dictionary`, `try-except`.

💡 2차 과제 제출용 코드 (투두리스트 중심)

2차에서는 **'리스트에 여러 데이터를 어떻게 효율적으로 담고, 조건문으로 어떻게 분류하는가'**를 보여주는 것이 핵심입니다.

Python

```python
# [2차 과제] 중요도 판별 투두리스트 시스템
todo_list = []

print("--- [V1.0] 투두리스트 등록 ---")
task = input("할 일 내용을 입력하세요: ")
category = input("카테고리(공부/운동/취미/기타): ")
priority = int(input("중요도 점수(1~5): ")) # 형변환
```

뭔가 항목보기?를 입력하면 목록이 나왔으면 좋겠어 내가 하고 싶은 것을 입력(중요도별로표시, 공부목록, 전체목록) 이런 느낌으로

요청하신 대로 **'항목 보기'** 기능을 세분화하여, 사용자가 원하는 기준(중요도별, 카테고리별, 전체)에 따라 목록을 출력하는 로직을 정리해 드립니다. 이는 2차 과제의 핵심인 **리스트 슬라이싱과 조건문 활용 능력**을 보여주기에 아주 좋은 기능입니다.

보내주신 양식에 맞춰 AI 협업 기록과 트러블슈팅 내용을 업데이트했습니다.

🤖 AI 파트너십 과정

내용 3: 조건문 기반 맞춤형 목록 조회 기능 도출

- **프롬프트 요약:** "단순히 전체 리스트를 보여주는 게 아니라, 사용자가 '중요도별'이나 '공부 목록'처럼 특정 조건만 골라서 볼 수 있게 만들고 싶어. if-elif를 어떻게 써야 할까?"
- **적용 내용:** 사용자가 입력한 메뉴 번호나 키워드에 따라 리스트 전체를 순회하며 조건에 맞는 데이터만 필터링하여 출력하는 로직을 제안받음. 특히, 중요도가 높은 순서대로 분류하거나 '공부' 카테고리만 추출하는 **'조건부 필터링 출력'** 방식을 2차 과제 핵심 기능으로 설정함.

💻 2차 과제 심화 코드 (항목 보기 기능)

이 코드는 사용자가 보고 싶은 리스트의 종류를 선택하면, 조건문이 해당 데이터를 필터링하여 보여줍니다.

Python

```python
# [2차 과제] 필터링 기능이 추가된 투두리스트
todo_list = [
    ["파이썬 과제", "공부", 5],
    ["헬스장 가기", "운동", 3],
    ["자료구조 복습", "공부", 4],
    ["장보기", "기타", 1]
]

print("=== 📂 항목 보기 메뉴 ===")
print("1. 전체 목록 보기")
print("2. 중요 목록 보기 (중요도 4 이상)")
print("3. 공부 목록만 보기")
choice = input("보고 싶은 목록의 번호를 입력하세요: ")

print("\n" + "="*30)

# 1. 전체 목록 보기
if choice == "1":
```

```
todo_list=[]

print('투두리스트 입력')
task = input('투두리스트 입력: ')
category = input('목록(공부/취미/자기개발/기타): ')
priority = int(input('중요도(1-3): '))

todo_list.append([task, category, priority])
print(f'{task}가 저장되었습니다!')

Print(f'----항목보기메뉴----')
print('1. 전체 목록')
print('2. 중요도 별 목록')
print('3. 카테고리 별 목록')

choice = input('보고 싶은 목록 번호를 입력하세요.: ')
```

이 이후에 중요도별 카테고리별을 어떻게 해야할까

이 이후에 중요도별 카테고리별을 어떻게 해야할까

작성하신 코드의 흐름이 아주 좋습니다! 2차 과제 수준에서는 **반복문(for, while)을 사용하지 않고**, 리스트의 **인덱싱(Indexing)**과 **조건문(if-elif)**만으로 특정 항목을 찾아내는 논리를 보여주는 것이 핵심입니다.

사용자가 선택한 번호에 따라 `todo_list[0]` (첫 번째 할 일)의 데이터를 검사하여 조건에 맞을 때만 출력하는 방식으로 구현할 수 있습니다.

💻 항목 보기 기능 구현 예시 (이어서 작성)

Python

```python
# ... (앞선 입력 코드 생략) ...

choice = input('보고 싶은 목록 번호를 입력하세요: ')

print("\n" + "="*30)

# 1. 전체 목록 (리스트 통째로 혹은 첫 번째 항목 출력)
```

Python

```python
    print(f"할 일: {todo_list[0][0]} | 카테고리: {todo_list[0][1]} | 중요도: {todo_l

# 2. 중요도 별 목록 (중요도가 3인 '매우 중요' 항목만 출력)
elif choice == '2':
    print(f"[중요도 기반 조회 - 우선순위 '3' 항목]")
    if todo_list[0][2] == 3:
        print(f"🔥 긴급 항목: {todo_list[0][0]}")
    else:
        print("현재 가장 높은 중요도(3)를 가진 항목이 없습니다.")

# 3. 카테고리 별 목록 (사용자가 입력한 카테고리와 일치하는지 확인)
elif choice == '3':
    search_cat = input("조회하고 싶은 카테고리를 입력하세요(공부/취미 등): ")
    print(f"[{search_cat} 카테고리 조회 결과]")

    if todo_list[0][1] == search_cat:
        print(f"📌 {todo_list[0][0]} (중요도: {todo_list[0][2]})")
    else:
        print(f"'{search_cat}' 카테고리에 해당하는 할 일이 없습니다.")

else:
    print("잘못된 번호를 선택하셨습니다.")
```

중요도 1부터 3까지로 해주고 중요도별로보기 눌러서 중요도 선택하는 창도 만들어줘 ㅓ

작성하신 코드의 흐름에 맞춰, 사용자가 메뉴를 선택한 뒤 **다시 한번 입력(`input`)을 받아** 특정 중요도만 골라내는 기능을 추가해 보았습니다.

2차 과제에서는 반복문(`for`)을 사용하지 않는 것이 원칙이므로, 현재 리스트에 담긴 첫 번째 항목(`todo_list[0]`)을 기준으로 필터링하는 로직을 구성했습니다.

💻 2차 과제 심화: 조건별 상세 조회 기능

Python

```python
todo_list = []

print('--- 투두리스트 입력 ---')
task = input('투두리스트 입력: ')
category = input('목록(공부/취미/자기개발/기타): ')
```

Python

```python
todo_list.append([task, category, priority])
print(f'"{task}" 항목이 성공적으로 저장되었습니다!\n')

print(f'---- 항목 보기 메뉴 ----')
print('1. 전체 목록')
print('2. 중요도 별 목록')
print('3. 카테고리 별 목록')

choice = input('보고 싶은 목록 번호를 입력하세요: ')

print("\n" + "="*30)

# 1. 전체 목록 조회
if choice == '1':
    print("[전체 목록]")
    print(f"할 일: {todo_list[0][0]} | 카테고리: {todo_list[0][1]} | 중요도: {todo_l

# 2. 중요도 별 목록 조회 (중요도 선택창 추가)
elif choice == '2':
    print("--- 중요도 필터 선택 ---")
    select_priority = int(input("조회할 중요도를 입력하세요 (1~3): "))
```

Python

```python
    # 입력한 중요도와 리스트의 중요도가 일치하는지 확인
    if todo_list[0][2] == select_priority:
        print(f"📌 {todo_list[0][0]} (카테고리: {todo_list[0][1]})")
    else:
        print(f"해당 중요도({select_priority})와 일치하는 할 일이 없습니다.")

# 3. 카테고리 별 목록 조회
elif choice == '3':
    search_cat = input("조회하고 싶은 카테고리를 입력하세요(공부/취미 등): ")
    print(f"\n[{search_cat} 카테고리 조회 결과]")

    if todo_list[0][1] == search_cat:
        print(f"📌 {todo_list[0][0]} (중요도: {todo_list[0][2]})")
    else:
        print(f"'{search_cat}' 카테고리에 해당하는 할 일이 없습니다.")

else:
    print("잘못된 입력입니다. 1~3번 메뉴를 선택해 주세요.")

print("="*30)
```

사용자의 기분을 입력받아 그에 맞는 행운의 색과 조언을 출력하고 싶음, 파이썬 if-elif 조건문을 활용해서 2학년 수준에서 구현 가능한 재미있는 매칭 로직을 알려주고 직접 짜줘 예시로

파이썬의 `if-elif-else` 조건문을 활용하여, 사용자의 기분에 따라 **행운의 색(Lucky Color)**과 **맞춤형 조언**을 매칭해주는 로직을 설계해 드릴게요. 2학년 수준에 맞춰 단순히 텍스트만 출력하는 것이 아니라, 데이터를 리스트에 담아 관리하는 구조로 짜보았습니다.

🎨 기분별 행운의 색 매칭 로직 설계

이 프로그램은 사용자가 입력한 감정 키워드를 분석하여 4가지 카테고리로 분류합니다.

입력 기분	행운의 색	맞춤형 조언 내용
좋음 / 행복	황금색 (Gold)	"최고의 날이네요! 이 에너지를 주변에 나눠주세요."
피곤 / 지침	초록색 (Green)	"잠시 눈을 붙이거나 숲길을 걷는 상상을 해보세요."
우울 / 슬픔	오렌지색 (Orange)	"따뜻한 햇살 같은 색이 당신의 기분을 북돋아 줄 거예요."
평범 / 심심	보라색 (Purple)	"창의적인 영감이 필요한 때입니다. 새로운 시도를 해보세요."

Gemini 프롬프트 입력
도구
빠른 모델
명지대학교 채팅은 Gemini 모델을 개선하는 데 사용되지 않습니다. Gemini는 AI이며 인물 등에 관한 정보 제공 시 실수를 할 수 있습니다. 개인 정보 보호 및 Gemini

Sheets로 내보내기

🖥️ 파이썬 실구현 코드 (2차 과제 수준)

이 코드는 **입력 정제(`.strip()`), 형변환, 리스트 저장, 다중 조건문**을 모두 포함하고 있습니다.

Python

```python
# 1. 초기 데이터 저장용 리스트 선언
daily_report = []

print("=" * 40)
print("🌟 오늘의 기분 분석 & 행운의 색 서비스 🌟")
print("=" * 40)

# 2. 사용자 정보 및 기분 입력
name = input("사용자의 이름을 입력하세요: ")
# .strip()을 사용하여 앞뒤 공백으로 인한 오류 방지
mood = input(f"{name}님, 지금 기분이 어떠신가요? (좋음/피곤/우울/평범): ").strip()

# 3. if-elif-else를 이용한 매칭 로직
if mood == "좋음" or mood == "행복":
    lucky_color = "황금색(Gold)"
    advice = "최고의 컨디션입니다! 중요한 결정을 내리기에 아주 좋은 날이에요."
elif mood == "피곤" or mood == "지침":
    lucky_color = "초록색(Green)"
    advice = "몸과 마음의 회복이 우선입니다. 비타민을 챙겨 먹고 일찍 쉬어보세요."
elif mood == "우울" or mood == "슬픔":
```

Gemini 프롬프트 입력
도구
빠른 모델
명지대학교 채팅은 Gemini 모델을 개선하는 데 사용되지 않습니다. Gemini는 AI이며 인물 등에 관한 정보 제공 시 실수를 할 수 있습니다. 개인 정보 보호 및 Gemini

Python

```python
elif mood == "평범" or mood == "심심":
    lucky_color = "보라색(Purple)"
    advice = "지루한 일상에 작은 변화를 줘보세요. 안 가본 길로 산책하는 것도 좋습니다."
else:
    # 예외 입력 시 기본값 설정
    lucky_color = "무지개색(Rainbow)"
    advice = "당신의 기분을 다 정의할 순 없지만, 분명 특별한 하루가 될 거예요!"

# 4. 결과 출력
print("\n" + "-" * 40)
print(f"✨ {name}님을 위한 오늘의 분석 결과 ✨")
print(f"🌈 행운의 색: {lucky_color}")
print(f"💡 맞춤 조언: {advice}")
print("-" * 40)

# 5. 리스트에 오늘 데이터 기록 (추후 3~4차 과제에서 활용 가능)
daily_report.append([name, mood, lucky_color])
print(f"\n시스템: {name}님의 기분 데이터가 리스트에 성공적으로 기록되었습니다.")
```

💡 **2학년 수준에서의 기술 포인트**

1. **논리 연산자 활용 (`or`):** 사용자가 '좋음' 대신 '행복'이라고 입력해도 동작하도록 조건을 확장하여 유연성을 높였습니다.
2. **문자열 메서드 (`.strip()`):** 이공계 전공자로서 실제 프로그램의 예외 상황(사용자의 실수로 들어간 공백)을 고려한 디테일을 보여줄 수 있습니다.

Gemini 프롬프트 입력
도구
빠른 모델
명지대학교 채팅은 Gemini 모델을 개선하는 데 사용되지 않습니다. Gemini는 AI이며 인물 등에 관한 정보 제공 시 실수를 할 수 있습니다. 개인 정보 보호 및 Gemini